

Name:	Meet Hemant Ghodke
Project:	Designing an embedded calculator

Circuit:



Figure 1: Circuit arrangement

Circuit Connections:

- An LCD screen is attached to the main AVR board.
- Button 1 → PBo
- Button 2 → PCo
- Potentiometer 1 → PC1
- Potentiometer 2 → PC2

Working of the project:

A user is supposed to select two numbers in between 0 and 999 via the potentiometers. The first potentiometer (connected to PC1) is for the first number and the second potentiometer (connected to PC2) is for the second number. Both the potentiometers have the same working where the value increases when turned clockwise and the value decreases when turned anticlockwise.

The basic mathematical operations (+, -, *, \) will appear on the side of the numbers. The first row on the LCD screen will have number 1 which is chosen by the user, the sum of both the numbers and the difference of both the numbers whereas the second row on the LCD screen will display the second number chosen by the user, the product of the two numbers and the division of two numbers.

On pressing the 1st button (connected to PBo), the user will be able to see the squares of both the numbers, where the square of 1st number will be displayed in the first row and the square of 2nd number will be displayed on the second row.

On pressing the 2nd button (connected to PCo), the user will be able to get the hexadecimal values of the numbers.

There won't be any necessity to re-flash the program since after this, the user can simply over-write and change the values and the results that would appear would be in correspondence to the new values.

Procedure and Elaboration of the project:

- Firstly, the potentiometers were chosen as the way to increase or decrease the values because they are much more efficient than repeatedly tapping the buttons.
- The variables value_1 and value_2 would be the ones which would be changed by the potentiometers. The 'sum' variable would contain the sum of both the values (initially declared as 0), the 'product' variable would contain the product of both the values (initially declared as 1), the 'difference' variable would contain the difference between the two values, sqr1 is the square of value_1, sqr2 is the square of value_2. Following is the way to declare the mentioned variables.

```
uint32_t value_1 = 0, value_2 = 0;
uint32_t sum = 0 ;
uint32_t product = 1;
int32_t difference = 0;
uint32_t sqr1 = 1, sqr2 = 1;
char line[16];
```

Figure 2 : Declarations of the variables used throughout the program

- The two buttons connected to PBo and PCo are supposed to be the interrupts. We need to set them as inputs and then activate their pullups like demonstrated in the following code.

```

DDRB &= ~(1<<DDB0);
DDRB &= ~(1<<DDC0);
PORTB |= (1<<PB0);
PORTC |= (1<<PC0);

```

Figure 3 Setting them as inputs and enabling the pullups of PBo and PCo

- PCICR |= (1<<PCIEo)|(1<<PCIE1); is then written where PCICR is the Pin Change Interrupt Control Register and writing a 1 to PCIEo (Pin Change Interrupt Enable o) and PCIE1 (Pin Change Interrupt Enable 1) , we enable them from ports PORTB and PORTC.
- PCMSKo (Pin Change Mask Register o) and PCMSK1 (Pin Change Mask Register 1) enable PBo and PCo.
- At the end, sei() is written to enable the global interrupts.
- Next since the potentiometers are being used, we would like to enable the ADC conversion. To do that, the following lines of codes are necessary.

```

ADMUX = (1<<REFS0); //5V reference voltage
ADCSRA = (1<<ADEN) | (1<<ADPS2) | (1<<ADPS1); //Enable the ADC with 64 prescaler division factor

```

Figure 4 Enabling the ADC and setting 5V as reference.

- In the main loop, we first set the ADC channel 1 (ADMUX |= (1<<MUX0)). We start the ADC conversion by writing ADCSRA |= (1<<ADSC). We wait for the conversion to happen and when it does the ADCW value is stored in value_1. Due to the LCD allowing only 16 characters on each row, the range of numbers chosen by the user are limited from 1 to 999. ADMUX &= ~(1<<MUX0) resets the MUX0 bit.
- The same logic is followed when we want to set the ADC channel 2. Following code helps to understand these statements.

```

ADMUX |= (1<<MUX0);
ADCSRA |= (1<<ADSC);
while(ADCSRA & (1<<ADSC));
value_1 = ADCW;
if(value_1>1000)
{
    value_1=999;
}
ADMUX &= ~(1<<MUX0);
ADMUX |= (1<<MUX1);
ADCSRA |= (1<<ADSC);
while(ADCSRA & (1<<ADSC));
value_2 = ADCW;
if(value_2>1000)
{
    value_2=999;
}
else if(value_2 < 0)
{
    value_2 = 1;
}
ADMUX &= ~(1<<MUX1);

```

Figure 5 Setting Channels 1 and 2 as ADC inputs

- Since in the main loop , we are only dealing with only basic arithmetic operations (+ , - , * , /), we write the following formulae in the code.

```
sum = value_1 + value_2;
difference = value_1 - value_2;
product = value_1*value_2;
```

Figure 6 Arithmetic Operations

- It is important to note that difference variable must be signed because it can be negative.

```
lcd_clear();
lcd_setcursor(0,1);
sprintf(line,"%3ld +%4ld - %3ld",value_1,sum , difference);
lcd_string(line);
lcd_setcursor(0,2);
sprintf(line,"%3ld *%6ld /%ld.%ld",value_2,product ,value_1/value_2 ,((value_1*value_2)*10)/value_2);
lcd_string(line);
```

Figure 7 Command to print the values on LCD

- From the above code, since uint32_t is used, the print specifier is %ld. The constant before 'ld' determines how much space will it take on the LCD.
- We are interested to represent the quotient with a decimal place. So, the main number is simply the quotient when both numbers are divided but for getting the decimal part, we could find the remainder after the first division step between the numbers, multiply that remainder by 10 and then divide the product by the second number. This way we can obtain the decimal without using float variables or modifying the makefile.
- It is necessary to include delay in the main loop towards the end so that the LCD screen doesn't fluctuate, and the values don't keep changing themselves.
- **When Button 1 is pressed, the first interrupt is called. This displays the squares of both the values selected by the user.**
- In the interrupt, firstly we have to give the formula to sqr1 and sqr2 on basis of value_1 and value_2.
- And then we simply print the result on the LCD screen with the help of the following codes:

```
ISR(PCINT0_vect)
{
sqr1 = value_1*value_1;
sqr2 = value_2*value_2;
lcd_clear();
sprintf(line, "SQR1: %7ld",sqr1);
lcd_setcursor(0,1);
lcd_string(line);
sprintf(line, "SQR2: %7ld",sqr2);
lcd_setcursor(0,2);
lcd_string(line);
_delay_ms(1500);
}
```

Figure 8 When Button 1 is pressed, this interrupt is called.

- On pressing the second button , the second interrupt is called and the hexadecimal values are displayed.

```
ISR(PCINT1_vect)
{
    lcd_clear();
    sprintf(line, "No.1 HEX: %x",value_1);
    lcd_setcursor(0,1);
    lcd_string(line);
    sprintf(line, "No.2 HEX: %x",value_2);
    lcd_setcursor(0,2);
    lcd_string(line);
    _delay_ms(1500);
}
```

Figure 9 When Button 2 is pressed, this interrupt is called.

OBSERVATION AND TESTING THE PROGRAM:

- Let the numbers given by user be : 582 (value_1) and 209 (value_2).
The basic operation results are showed on the right side.



Figure 10 User choosing two numbers and the basic arithmetic operations are performed.

- After clicking the 1st button , the LCD should display the squares of each numbers. (582 and 209)

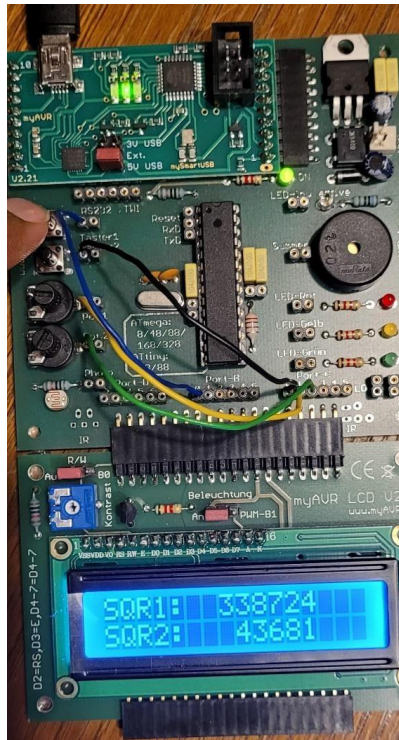


Figure 11 The squares of both the numbers displayed

- After clicking the second button, the LCD should display the hexadecimal numbers of 582 and 209.



Figure 12 Hexadecimal Representation of numbers chosen

